PRACTICAL LAB REPORT

Local AI with Ollama

Student: Firdaous Boulben
Academic Year: 2026–2027

---

1. Introduction

This report presents the main concepts and results obtained during the practical lab on Local AI with Ollama. It covers the fundamentals of local language models, their integration into Python applications, and the use of techniques such as Retrieval-Augmented Generation (RAG). It also discusses the experimental comparison of two local models, qwen3.5 and gemma3, based on their performance and resource consumption. Finally, the report examines the advantages, limitations, and security considerations of running AI models locally.

2. Questions and Analysis

Q1. What is Local AI?

Local AI refers to artificial intelligence models that run directly on a personal computer or another local device, rather than depending on remote servers to process requests. In this case, the model uses the device's own computing resources, such as its CPU, GPU, and RAM. Once the necessary models have been downloaded, many local AI applications can operate without an internet connection, offering users greater control over how and where their data is processed.

Q2. What is the difference between a Cloud LLM and a Local LLM?

The primary difference between a Cloud LLM and a Local LLM lies in where they are hosted and executed. Cloud LLMs run on remote servers, where they can benefit from powerful hardware and provide access to larger models. However, they generally require an internet connection and may involve usage fees. Local LLMs, on the other hand, run directly on the user's computer, giving users greater control over their data and potentially better privacy, since prompts and documents can remain on the device. Their performance, however, depends on the computer's hardware and available memory.

Q3. What role does Ollama play?

Ollama provides a simple way to download, run, and manage language models locally. It handles the model execution process and exposes an HTTP API that applications can use to send prompts and receive generated responses.

In this lab, Ollama served as the link between the local language models and the Python scripts. As a result, it was possible to test different models and develop AI-powered applications without having to implement the model execution process from scratch.

Q4. What is model quantization?

Model quantization is a technique that reduces the precision used to represent a model's weights. For example, instead of using higher-precision values, a model can use 4-bit values, which require less memory. This makes it possible to run models on computers with limited resources and may also improve response speed. However, it can also affect the quality of the generated responses, depending on the model and quantization method.

Q5. Why does model size affect memory consumption?

Model size affects memory consumption because larger models contain more parameters, which require more RAM to store and process. As a result, the larger the model, the more memory it generally needs to run, while smaller models can operate with fewer resources.

Q6. Why can a smaller model sometimes be preferable?

A smaller model can be preferable because it generally requires less RAM and storage and may generate responses faster on a computer with limited resources. It can also be easier to run without a dedicated GPU.

Q7. What are the advantages of local inference?

Local inference offers several advantages:

- **Privacy:** Prompts and documents can remain on the local device.

- **Offline access:** The model can operate without an internet connection after installation and download.
- **Control:** Users can choose models and customize their applications.
- **Reduced API costs:** Local inference avoids per-request charges from external model APIs, although hardware and electricity still have costs.
- **Integration:** Local models can be connected to Python scripts, document retrieval systems, and web applications.

Q8. What are the disadvantages of local inference?

Local inference has several disadvantages:

- **Hardware requirements:** Running large models may require a powerful computer with enough RAM and a dedicated GPU, which can be expensive.
- **Performance limitations:** The available hardware limits which models can run efficiently and how quickly they generate responses.
- **Complex setup:** Installing the necessary software and fixing configuration issues can be difficult for some users.
- **Resource consumption:** Running a model can slow down other applications, drain a laptop's battery, and cause the computer to heat up.
- **Manual maintenance:** Users are responsible for updating models, managing storage, and maintaining the local environment.

Q9. Explain the architecture of your Python application.

The application uses Python as the main programming language and Ollama as the local model runtime. The final mini-project also uses Flask to provide a browser-based interface.

When a user submits a question, the Flask application receives it through an HTTP route. The retrieval component searches the local course documents and selects relevant information, which is added to the prompt as context before it is sent to Ollama. The selected local model processes the request and returns a response. The Flask application then sends the answer back to the browser for display, along with information such as the retrieved sources and response time.

Overall, the project combines a user interface, Python application logic, document retrieval, and local language generation. A separate console chatbot provides a simpler way to interact with Ollama directly through the terminal.

Q10. Explain the role of the REST API.

A REST API allows different software components to communicate through HTTP requests and responses. In this lab, Python sends requests to Ollama's local API, including the model name and the prompt or conversation messages. These requests are generally formatted as JSON. Ollama processes the input and returns the generated response in a structured format, which Python can then extract and use in a script or web application.

This approach separates the application logic from the model execution process and makes it easier to integrate local AI into different software systems.

Q11. What is RAG?

Retrieval-Augmented Generation (RAG) is an approach that combines document retrieval with language generation. Instead of asking a language model to answer a question using only its existing knowledge, the system first searches a collection of documents for relevant information. The retrieved content is then included in the prompt, helping the model generate an answer based on the supplied context.

In this lab, the RAG system searched local AI and course documents before generating responses. This approach can make answers more relevant to the provided material and help the system respond to questions about specific documents. However, the quality of the results depends on whether the correct information is retrieved and how accurately the model uses it.

Q12. What is the difference between an LLM and a multimodal LLM?

A traditional text-based LLM primarily processes and generates text. A multimodal LLM can process multiple types of input, such as text and images, depending on the model's capabilities.

For example, in the vision exercise (Part 14), an image was provided to a model that supports image input. The model could then describe visible objects and elements of the scene. This

shows how multimodal models can be used for tasks such as image description, visual question answering, and document analysis.

Q13. Compare two local models experimentally.

In Part 6, qwen3.5 and gemma3 were tested on a prime-checking programming task. Both models passed the tests. However, their generated implementations differed: qwen3.5 used an optimized approach involving an integer square root and skipped even divisors, while gemma3 produced a simpler and more readable implementation.

In Part 16, the models were also compared using resource and performance measurements collected on my computer. The results are presented below.

Table 1: Experimental comparison of qwen3.5 and gemma3

Metric	***qwen3.5***	***gemma3***
Disk size (GB)	6.55	3.35
Loaded size (GB)	6.19	2.89
VRAM (GB)	0.0	0.0
Load time (s)	337.19	167.74
Response time (s)	618.70	319.55
Generated tokens	694	818
Generation speed (tokens/s)	3.47	6.84
System RAM used (%)	94.0	73.1

The results show that gemma3 was more resource-efficient on my computer. Its disk size and loaded size were smaller, and its measured system RAM usage was lower. It also generated responses faster in this test, with a generation speed of 6.84 tokens per second compared with 3.47 tokens per second for qwen3.5.

Qwen3.5 consumed more resources and took longer to generate responses during the benchmark. However, these measurements alone do not determine which model performs well overall. The quality, correctness, and suitability of the responses must also be considered.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

Based on the experimental results, I would choose gemma3 for a computer with limited RAM. Its loaded size was 2.89 GB, compared with 6.19 GB for qwen3.5, and its measured system RAM usage was 73.1%, compared with 94.0%. Gemma3 also achieved a higher generation speed and a shorter response time in the benchmark.

These results make it a more practical option when memory consumption and speed are priorities. However, I would still evaluate its response quality on the intended tasks before deployment. The best model should provide an acceptable balance between accuracy, resource requirements, and performance.

Q15. What security risks exist when exposing a local LLM server to a network?

Exposing a local LLM server to a network can create security risks, particularly because Ollama's API has no authentication by default. If the service is configured to listen on all network interfaces, for example through OLLAMA_HOST=0.0.0.0, the service may be accessible to other devices on the network.

This can allow unauthorized users to consume system resources, causing performance problems or resource exhaustion. Depending on the application and its configuration, network access may also expose model functionality or information included in prompts and retrieved documents.

Another risk is prompt injection. A malicious document retrieved by a RAG system may contain instructions intended to manipulate the model into ignoring its intended rules or revealing information from its context.

To reduce these risks, access should be restricted using a firewall or a reverse proxy with authentication and HTTPS/TLS encryption.

3. Conclusion

This lab provided practical experience with local AI by demonstrating how Ollama, Python, and Flask can be combined to build a functional chatbot. It also explored key concepts such as prompt engineering, RAG, structured output, and multimodal models.

The benchmark results showed that gemma3 was faster and consumed fewer resources than qwen3.5 on the tested computer, making it a more suitable option for systems with limited RAM, provided that its response quality is sufficient.

Overall, the project highlighted the benefits of local AI, including greater control over data and reduced dependence on cloud services. However, successful deployment also requires balancing performance, memory usage, and response quality while applying appropriate security measures.